



DESIGN (E) 314
TECHNICAL REPORT

Smart Prepaid Energy Controller

Author:
Kyle Cohead

Student Number:
25964917

May 17, 2025

Plagiaatverklaring / Plagiarism Declaration

1. Plagiaat is die oorneem en gebruik van die idees, materiaal en ander intellektuele eiendom van ander persone asof dit jou eie werk is.

Plagiarism is the use of ideas, material and other intellectual property of another's work and to present is as my own.

2. Ek erken dat die pleeg van plagiaat 'n strafbare oortreding is aangesien dit 'n vorm van diefstal is.

I agree that plagiarism is a punishable offence because it constitutes theft.

3. Ek verstaan ook dat direkte vertalings plagiaat is.

I also understand that direct translations are plagiarism.

4. Dienooreenkomstig is alle aanhalings en bydraes vanuit enige bron (ingesluit die internet) volledig verwys (erken). Ek erken dat die woordelike aanhaal van teks sonder aanhalingstekens (selfs al word die bron volledig erken) plagiaat is.

Accordingly all quotations and contributions from any source whatsoever (including the internet) have been cited fully. I understand that the reproduction of text without quotation marks (even when the source is cited) is plagiarism.

5. Ek verklaar dat die werk in hierdie skryfstuk vervat, behalwe waar anders aangedui, my eie oorspronklike werk is en dat ek dit nie vantevore in die geheel of gedeeltelik ingehandig het vir bepunting in hierdie module/werkstuk of 'n ander module/werkstuk nie.

I declare that the work contained in this assignment, except where otherwise stated, is my original work and that I have not previously (in its entirety or in part) submitted it for grading in this module/assignment or another module/assignment.



Handtekening / *Signature*

25964917

Studentenommer / *Student number*

KL Cohead

Voorletters en van / *Initials and surname*

17 / 05 / 25

Datum / *Date*

Abstract

This report presents an in depth description of how a Smart Prepaid Energy Controller is developed. The system enables accurate energy monitoring and load regulation, specifically tailored towards Airbnb rentals and student accommodation in Stellenbosch. The system incorporates prepaid electricity metering capabilities with RFID authentication for system owners. This report details the design, implementation, and evaluation of the system.

Contents

1	Introduction	1
2	System Description	1
3	Hardware Design and Implementation	2
3.1	Hardware Block Diagram and Description of Interaction	2
3.2	Power Supply	2
3.2.1	5V Regulator Circuit	2
3.2.2	3.3V Regulator Circuit	3
3.2.2.1	3.3V Regulator Power Dissipation	3
3.3	LEDs (Debug)	3
3.3.1	LED Series Resistors	4
3.3.2	LED Implementation	4
3.4	Voltage and Current Sensing Circuits	4
3.4.1	Conceptual System	4
3.4.2	Signal Conditioning	5
3.4.3	Component Selection	5
3.4.4	Circuit Design	6
3.5	Load Control Circuit	6
3.5.1	Circuit Design	6
3.5.2	Circuit Diagram	7
3.6	Keypad and Button	7
3.7	OLED	7
3.8	SD Card	8
3.9	RFID Reader	8
3.9.1	Decoupling Capacitor	8
4	Software Design and Implementation	8
4.1	Software Block Diagram and Description of Interaction	9
4.2	ADC, Setup and Sampling Data Management	9
4.2.1	Setup	10
4.2.2	Calibration	11
4.3	Analogue Measurements, Voltage, Current, and Phase	11
4.3.1	Measurement Process	11
4.3.2	Processing Voltage and Current	11
4.3.3	Processing Phase	12
4.4	Power Calculations, P, S, Q, and PF	12
4.5	UART Communications (Protocol, Data conversion)	12
4.5.1	UART Protocol	13
4.5.2	UART Timing	13
4.6	Keypad Driver Setup and Button Bounce Handling	14
4.6.1	Button Bounce Handling	14
4.7	OLED Low-Level Interface	14
4.8	FSM Control and Logic (Inputs: Keypad, UART. Outputs: LEDs, Load, OLED)	14
4.9	Energy Consumption Calculations + RTC	15
4.10	SD Card Interface and Logging	15
4.10.1	Low-level interface	15
4.10.2	Logging	15
4.11	RFID Interface and Authentication	16

5	Measurements and Results	16
5.1	Power supply	17
5.1.1	5V Regulator	17
5.1.2	3.3V Regulator	17
5.2	Voltage and Current Circuits	18
5.2.1	Circuit Measurement Setup	18
5.2.2	Circuit Measurement Test Results	18
5.2.3	Circuit Measurement Conclusions	18
5.3	Load Circuit	19
5.4	Keypad and Button	19
5.5	UART Communications	20
5.6	V, I, Phase, P, S, Q (incl. Calibration)	20
5.6.1	Calibration	20
5.6.2	Voltage and Current RMS Measurements	21
5.6.3	Phase Measurements	21
5.6.4	Power Measurements	21
5.7	OLED (Functional)	21
5.8	SD Card (Functional)	22
5.9	RFID (Functional)	22
5.10	Menu Functionality	23
5.11	Full System (reliable, consistent, stress testing)	23
5.11.1	Circuit Clipping	23
5.11.2	UART Commands	24
5.11.3	3.3V Supply Rail Noise	24
5.11.4	User LEDs	24
5.11.5	Circuit Performance	25
6	Conclusions	25
6.1	Non-compliance	25
6.2	Possible Improvements	25
7	Appendix	I

List of Figures

1	System Diagram	1
2	Hardware Block Diagram	2
3	5V Power Supply Circuit Diagram	2
4	3.3V Power Supply Circuit Diagram	3
5	Voltage and Current Measurement (Conceptual) System Diagram	4
6	Gain and Offset Circuit Diagram	6
7	Relay Circuit Diagram	7
8	Active High Configuration	7
9	Software Block Diagram	9
10	ADC Sampling Flow Chart	10
11	Phase Calculation Pseudo Code	12
12	Power Triangle	12
13	UART Tx and Rx Pseudo Code	13
14	Software Button Debouncing Pseudo Code	14
15	OLED Display FSM	15
16	SD Card Logging Pseudo Code	16
17	RFID Authorisation Psuedo Code	16
18	5V Regulator Measurement Setup	17
19	3.3V Regulator Measurement Setup	17

20	Circuit Measurement Setup	18
21	Load Off	19
22	Load On	19
23	Button Bounce	19
24	Keypad Bounce	19
25	Power-up Student Number Transmission	20
26	TX and RX Signals	20
27	OLED Display Functionality 1	22
28	OLED Display Functionality 2	22
29	SD Card Functionality with UART	22
30	Scanning Without Card	22
31	Scanned Card	22
32	Full FSM Traversal	23
33	Voltage Circuit Slight Clipping	23
34	Voltage Circuit Clipping	23
35	Incorrect UART Commands	24
36	3.3V Supply Rail (Antenna Off)	24
37	3.3V Supply Rail (Antenna On)	24
38	LED D4 4.5s Toggling	25
39	LED D5 500ms Toggling	25
40	Voltage Circuit Input (blue) and Output(yellow)	25
41	Project Schematic	I

List of Tables

1	GPIO Pin Assignments for Debug LEDs	4
2	Keypad Signal Connections	7
3	SD Card Module Signal Connections	8
4	RFID Module Signal Connections	8
5	ADC Initialisation Settings	10
6	DMA Initialisation Settings	10
7	UART TX Message fields - Log from board to PC	13
8	SD Card Log Format	16
9	5V Regulator Outputs	17
10	3.3V Regulator Outputs	17
11	Measurement Test Equipment	18
12	Voltage and Current Circuit Outputs and Gains	18
13	ADC Sampling Accuracy	18
14	RMS Voltage and Current Accuracy	21
15	Phase Measurement Accuracy	21
16	Power Measurement Accuracy	21
17	STM32 Pins Used and thier Configuration	II
18	OLED Display Screens for Various Input Combinations from Keypad.	III

List of Abbreviations

CPU Central Processing Unit

FSM Finite State Machine

MCU Micro Controller Unit

OLED Organic Light-Emitting Diode

List of Symbols

I Current

Ω Resistance

V Voltage

1 Introduction

The Smart Prepaid Energy Controller addresses the challenge of fair electricity usage and management in rental properties. The system allows owners to monitor energy consumption accurately, while providing tenants with clear electricity usage information. The system balances convenience with control by incorporating both prepaid functionality and owner override capabilities. This dual-purpose solution functions as both an electrical load control mechanism and an energy monitoring device, making it ideal for both rentals and personal guest visits, where energy costs must be managed efficiently.

2 System Description

The Smart Prepaid Energy Controller is a multifunctional device designed to monitor power usage and manage electrical loads through prepaid units. The system diagram is shown in Figure 1.

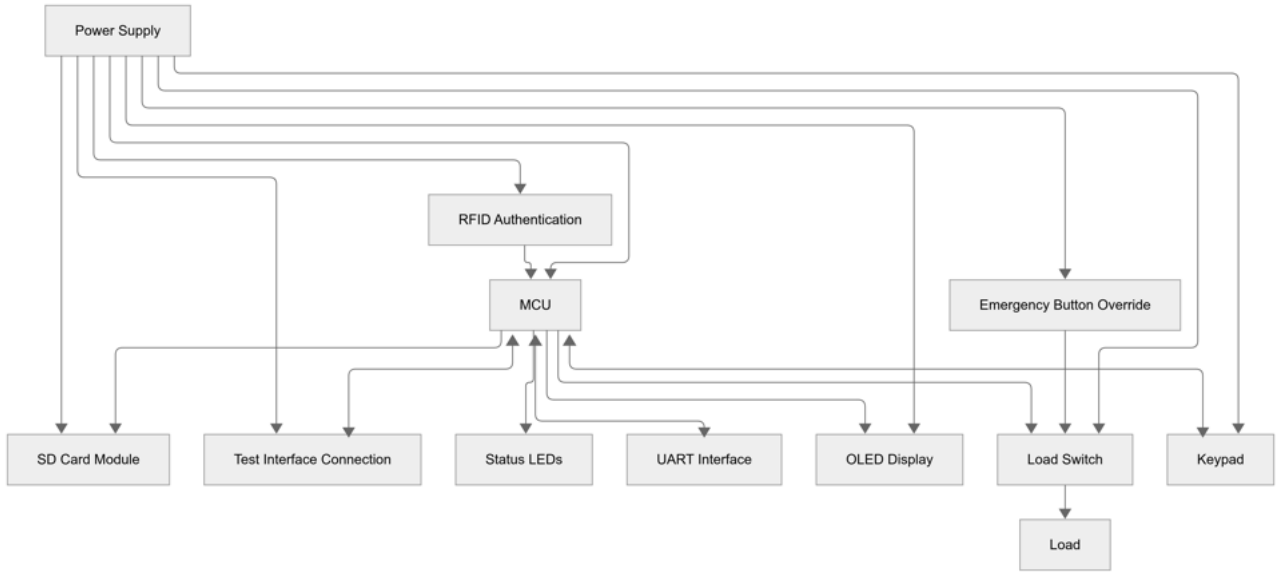


Figure 1: System Diagram

The system provides the following core functionalities:

1. **Power Supply:** The system must regulate supplies from a 9-12V battery or power source, providing 5V and 3.3V outputs. LED D6 confirms 5V power presence.
2. **Power Measurement:** The device measures load Voltage (V), Current (I), and phase (all to 5% accuracy), used to calculate apparent power, real power, and reactive power.
3. **User Interface:** An Organic Light-Emitting Diode (OLED) display presents system information while a keypad allows for user input. Status LEDs provide visual feedback on power consumption levels.
4. **Authentication System:** RFID technology enables secure authentication for system owners and validates prepaid energy credits for users.
5. **Load Control:** A relay switches the monitored electrical load on or off based on prepaid credit availability, with emergency override functionality.
6. **Data Management:** Power consumption data is logged to an SD card, ensuring recover-ability through power interruptions and enabling historical analysis.
7. **Communication:** A UART interface enables external connectivity for system monitoring and configuration via standardised commands.

8. Timekeeping: An integrated Real-Time Clock ensures accurate timestamping of all measurements and events.

The G0B1RE Micro Controller Unit (MCU) serves as the central coordination point, processing sensor inputs, managing the user interface, and controlling the relay based on authentication status and prepaid unit availability. This integrated approach enables reliable energy monitoring and access control.

3 Hardware Design and Implementation

3.1 Hardware Block Diagram and Description of Interaction

The system operates through coordinated hardware interactions which are all centered around the MCU. Power flows from a 9-12V source through voltage regulators (5V/3.3V) to all components, with LED D6 confirming power presence. The microcontroller processes inputs from voltage, current, and phase sensors connected to the electrical load, calculating power values within 5% accuracy (the TIC simulates the real world load). User interaction occurs through the keypad and OLED display, while status LEDs indicate usage levels. The RFID reader authenticates users and enables unit loading. Based on prepaid credits and authentication status, the microcontroller activates or deactivates the relay that controls the electrical load, with an emergency button to provide manual override. Events and measurements are timestamped by the RTC and logged to the SD card for data persistence. External communication is handled through the UART interface at 57600 baud and the TIC enables functionality testing. Figure 2 illustrates the hardware block diagram.

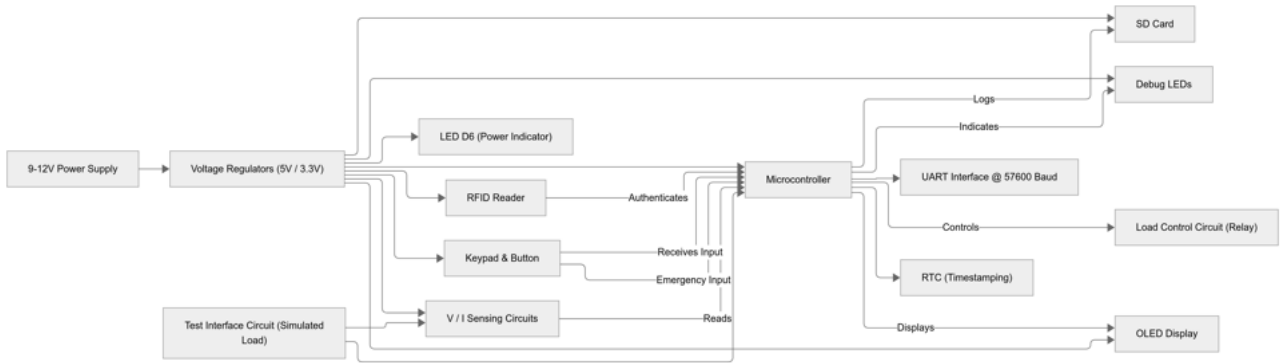


Figure 2: Hardware Block Diagram

3.2 Power Supply

The baseboard is supplied by a nominal 9V – 12V battery or power supply through a barrel jack which is connected to a 5V regulator circuit. The output of the 5V regulator serves as the input to a 3.3V regulator circuit.

3.2.1 5V Regulator Circuit

The 5V regulator used is the L7805CV, shown in Figure 3, which provides a stable 5V output from the 9V – 12V input [1]. It has three pins, discussed below, followed by a discussion of the components:

- Input (V_{in}): Unregulated input voltage.
- Ground (GND): Common reference point.
- Output (V_{out}): Regulated 5V output.

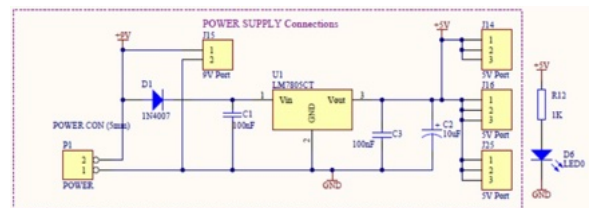


Figure 3: 5V Power Supply Circuit Diagram

- *D1*: Is a diode that protects the voltage regulator in case of reverse polarity.
- *C1*: Is a $0.1\mu F$ decoupling capacitor connected between the input pin (V_{in}) and the ground (GND) to stabilise the input voltage.
- *C2*: Is a $10\mu F$ decoupling capacitor connected between the output pin (V_{out}) and the ground (GND) to stabilise and reduce low-frequency ripples in the output voltage.
- *C3*: Is a $0.1\mu F$ decoupling capacitor connected in parallel with *C2* that suppresses high-frequency noise and oscillations that the regulator might generate.
- *D6*: Is an LED that turns on to indicate that the $5V$ regulated voltage is present. The current through the LED is limited by a $1k\Omega$ resistor (*R12*).

3.2.2 3.3V Regulator Circuit

The $3.3V$ regulator used is the L78L33 regulator, shown in Figure 4, provided in a TO-92 package. This regulator is designed to provide a fixed $3.3V$ output, from a minimum input voltage of $5V$ (and an absolute maximum of $30V$), with a maximum output current of $100mA$ (if suitable heat sinking hardware is provided) [2]. It has three pins, discussed below, followed by a discussion of the components:

- Input (V_{in}): $5V$ input voltage.
- Ground (GND): Common reference point.
- Output (V_{out}): Regulated $3.3V$ output.

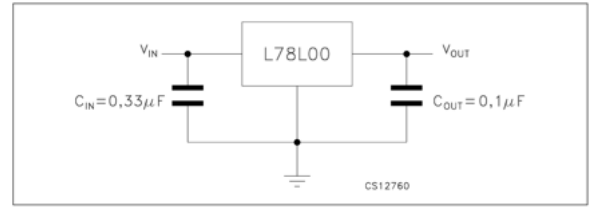


Figure 4: 3.3V Power Supply Circuit Diagram

- C_{IN} ($0.33\mu F$): A decoupling capacitor connected between the input pin (V_{in}) and ground (GND) to stabilise the input voltage and filter noise.
- C_{OUT} ($0.1\mu F$): A capacitor connected between the output pin (V_{out}) and ground (GND) to reduce voltage fluctuations and ensure a stable $3.3V$ output.

3.2.2.1 3.3V Regulator Power Dissipation

The power dissipation of the $3.3V$ regulator can be calculated using the following equation [2]:

$$P_D = (V_{in} - V_{out}) \times I_{load} \quad (1)$$

where:

- $V_{in} = 5V$ (input voltage),
- $V_{out} = 3.3V$ (regulated output voltage),
- $I_{load} = 50mA$ (sufficient maximum current drawn by the load).

Substituting these values:

$$P_D = (5V - 3.3V) \times 0.050A = 0.085W$$

The heat generated by the power dissipation of $85mW$ is minimal, so the regulator is thermally stable and no heat sink is required.

3.3 LEDs (Debug)

The baseboard provides space for four N0R28L32 LEDs [3] along with their respective series resistors, which are intended for debugging or user-defined indications. These LEDs are driven by four General Purpose Input/Output (GPIO) pins of the MCU. Since the LED circuit pins are floating and not connected to any other signals on the baseboard, they are manually wired to the desired GPIO pins.

3.3.1 LED Series Resistors

Each LED requires a current-limiting series resistor to prevent excessive current from being drawn by the MCU's output pins, which could otherwise be damaged. The maximum current sourced by a GPIO pin is $15mA$ [4]. The Resistance (Ω) value is determined using Ohm's law:

$$R = \frac{V_{\text{supply}} - V_f}{I} \quad (2)$$

where:

- $V_{\text{supply}} = 3.3V$ (MCU GPIO logic level),
- $V_f = 2V$ (LED forward voltage),
- $I = 3.33mA$ (Desired LED current to ensure safe GPIO operation).

Substituting in these values:

$$R = \frac{3.3V - 2V}{3.3mA} = 390\Omega$$

A 390Ω resistor is chosen to safely limit the current for each of the four LEDs. This ensures the current stays within the GPIO pin's maximum sourcing capability while maintaining sufficient brightness. Each LED is connected with its anode to a GPIO pin and its cathode to ground (GND) through the series resistor.

3.3.2 LED Implementation

The four LEDs are connected to four available GPIO pins on the MCU. These GPIO pins are configured as *output* mode in the firmware, allowing them to toggle the LEDs on or off according to system status, as given in the project design document [5]. Table 1 shows the pin assignments for the four LEDs:

LED	GPIO Pin	Indicates
D2	PA9	Load On/Off (ON = Load active)
D3	PA4	Energy units (Flashing = Low)
D4	PB0	Wh Ticker (1 flash per unit)
D5	PC7	Power usage (Flashing = High usage)

Table 1: GPIO Pin Assignments for Debug LEDs

3.4 Voltage and Current Sensing Circuits

3.4.1 Conceptual System

In Figure 5, the conceptual system diagram is shown [5]. The mains voltage is measured using a voltage divider and the current is measured using a current transformer with a measurement resistor to create a measured voltage. The transformed signals, both sinusoids with $0V$ offset (graph on the left), are processed by the board into signals (graph on the right) that can be measured by the MCU's Analogue to Digital Converter (ADC).

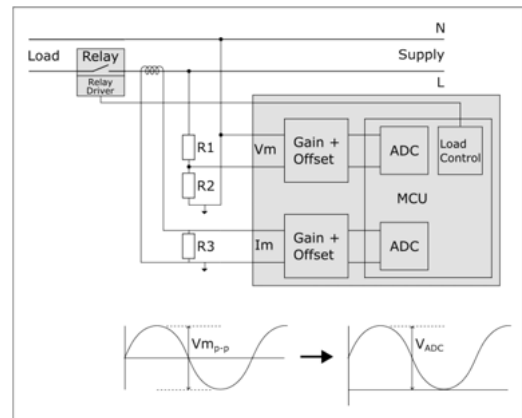


Figure 5: Voltage and Current Measurement (Conceptual) System Diagram

This system will simulate a real-world measurement scenario as shown in Figure 5. Thus, the output signals of the circuits not included in the gray boxes need to be specified.

- For the measurement of the mains voltage, a sinusoid ($V_{m_{pp}}$) with $2V_{pp}$ will represent a mains voltage of $230V_{RMS}$ [5]. This signal will be modified to adhere to the input requirements for the MCU's ADC.
- For the measurement of the current, a sinusoid ($V_{i_{pp}}$) with $2V_{pp}$ will represent a current of $15A_{RMS}$ [5]. This signal will also be modified to adhere to the input requirements for the MCU's ADC.

3.4.2 Signal Conditioning

An offset-and gain circuit, that will be realised using the MCP602 dual operational amplifier (op-amp) is designed to apply the correct offset and gain to the two input signals [6], to comply with the MCU's ADC.

The $2V_{pp}$ signals will produce negative voltage values that cannot be supplied to the ADC input, so a positive offset is required. Furthermore, this signal only uses $2V$ out of the ADC input range of $3.3V$, therefore a positive gain is required to make use of this larger ADC input range.

The op-amp circuit is designed with reference to the guidelines provided in [7]. The following voltages are selected, with the full-scale and zero-scale output voltages selected such that they fall within the ADC's input range of $0V - 3.3V$:

- Reference voltage: $V_{ref} = 3.3V$,
- Full-scale output voltage: $V_{outfs} = 3V$,
- Zero-scale output voltage: $V_{outzs} = 0.3V$,
- Full-scale input voltage: $V_{infs} = 1V$,
- Zero-scale input voltage: $V_{inzs} = -1V$.

This selection additionally ensures that the ADC is able to measure input voltages corresponding to $281V_{RMS}$, which is well above $253V_{RMS}$ (10% above $230V_{RMS}$), the maximum expected voltage over the load [5].

Using these values, the gain m and offset b are calculated as follows:

$$m = \frac{V_{outfs} - V_{outzs}}{V_{infs} - V_{inzs}} = \frac{3 - 0.3}{1 - (-1)} = 1.35, \quad (3)$$

$$b = V_{outzs} - m \times V_{inzs} = 0.3 - (1.35 \times (-1)) = 1.65. \quad (4)$$

3.4.3 Component Selection

The resistor values for the circuit are chosen or calculated based on the gain and offset requirements calculated above. After choosing $R_1 = 1k\Omega$ and $R_f = 1k\Omega$, the following equations are used to calculate R_2 and R_g [7]:

$$R_2 = \frac{V_{ref} \times R_1 \times m}{b}, \quad (5)$$

$$R_g = \frac{R_2 \times R_f}{m \times (R_1 + R_2) - R_2}. \quad (6)$$

Using these equations along with the chosen resistor values:

$$R_1 = 1k\Omega, \quad R_2 = 2.7k\Omega, \quad R_f = 1k\Omega, \quad R_g = 1176.471\Omega \approx 1176\Omega.$$

R_g is implemented using a series combination of a $1k\Omega$, a 120Ω , and a 56Ω resistor.

3.4.4 Circuit Design

Figure 6 shows the schematic diagram [7] for one op-amp circuit, both op-amp circuits are identically implemented, hardware wise.

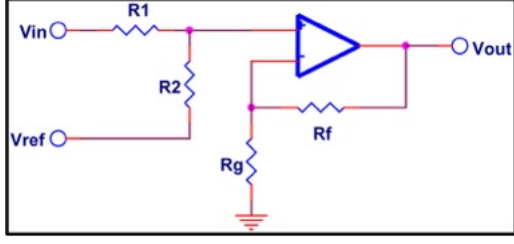


Figure 6: Gain and Offset Circuit Diagram

The voltages in Figure 6 are clarified below:

- V_{in} : Input voltage = $2V_{pp}$,
- V_{ref} : Stable reference voltage = $3.3V$,
- V_{out} : Circuit output/ADC input voltage.

3.5 Load Control Circuit

In a real-world implementation, the system would interface with high-voltage loads powered by the mains supply ($220V - 250V$). To safely simulate this interaction, a relay circuit is incorporated to control a low-voltage load, represented by an LED. This setup demonstrates the conceptual operation of switching a load on or off, without directly connecting to the mains supply.

The relay used is a J107 series relay (component number J1071AS5VDC36). This relay cannot be driven directly from the GPIO pins, due to the high coil current required [5]. Therefore, the 2N7000 N-Channel MOSFET is used as a driver to activate the relay.

3.5.1 Circuit Design

The relay's pickup voltage is $3.75V$ (max), and its release voltage is $0.5V$ (min), with operate and release times of $10ms$ each [8]. The relay coil, rated at $5V$ with a resistance of $69\Omega \pm 10\%$, is powered by a $5V$ supply connected to one coil terminal. The second coil terminal is connected to the drain of MOSFET, with the MOSFET's source grounded and its gate driven by GPIO pin PA9 operating at $3.3V$ logic [9].

This configuration allows the MCU to switch the relay coil, which consumes $0.36W$ [8] and draws approximately $72mA$, calculated using Ohm's law:

$$I = \frac{V}{R} \quad (7)$$

where:

- $V = 5V$ (relay supply),
- $R = 69\Omega$ (coil resistance).

Substituting in these values:

$$I = \frac{5V}{69\Omega} = 72mA$$

A flyback diode is placed across the coil to protect the MOSFET from voltage spikes during deactivation. The relay's common (*COM*) terminal is connected to a $3.3V$ supply, and the normally open (*NO*) contact switches the LED load to ground. When the relay is energised (GPIO high), the contacts close, completing the LED circuit. A current-limiting resistor is used in series with the LED, to prevent damage to the LED, and its resistance is calculated using equation 2, so a standard 390Ω resistor is used.

3.5.2 Circuit Diagram

Figure 7 shows the schematic diagram for the implemented relay circuit. The relay's activation simulates a real-world scenario where a load (e.g., an appliance) would be turned on or off based on power measurements, such as exceeding a threshold for apparent, real, or reactive power. In this prototype, the LED illuminates when the J107 relay is energised, providing a visual indication of the switching action.

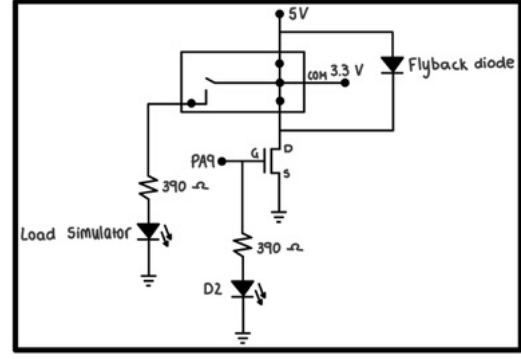


Figure 7: Relay Circuit Diagram

3.6 Keypad and Button

The push button is used to override the load control, which is useful during emergency situations. The load only switches on if there are energy units available on the system. If the load is active, a button press will always cause it to switch off. This allows a user to test the load control and to override the control if needed. The button is configured as active-high, as seen in Figure 8, and is de-bounced in software. The button is connected to GPIO pin PB4.

The keypad is the main user input interface for the system during normal operation. The keypad used is 4 (rows) x 3 (columns). The rows are connected as GPIO outputs and the columns are connected as GPIO inputs on the MCU. Since each key is effectively a button, inputs are de-bounced (in software) to prevent erroneous double-press events. In order to simplify the circuit and software, the internally available pull-down resistor configuration is used for the input pins. Table 2 lists the keypad's signal connections which enable communication with the MCU.

Module Signal	Connection
ROW1	GPIO pin PA11
ROW2	GPIO pin PB13
ROW3	GPIO pin PB10
ROW4	GPIO pin PC0
COL1	GPIO pin PC1
COL2	GPIO pin PA12
COL3	GPIO pin PB2

Table 2: Keypad Signal Connections

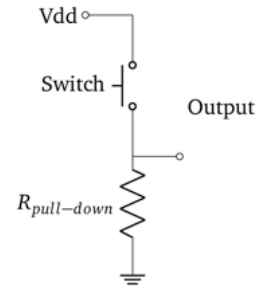


Figure 8: Active High Configuration

The rows and columns are connected in an active high configuration (pull-down resistor implemented in software) so that when a key is pressed, a high input is read and when no key is pressed, the input is read low. Figure 8 [5] shows how this is implemented. When the switch is closed (ON), the output is pulled to V_{dd} (HIGH) and when the switch is open (OFF), the output is pulled to ground (LOW).

3.7 OLED

The OLED display used in this project uses an I2C interface and has a resolution of 128 x 32. The diagonal screen size: 0.91" and the display area is 7mm x 25mm. The graphic driver IC is the SSD1306ide and the OLED's input voltage is 2.5V-5.5V [5].

The OLED display is powered by the 3.3V supply, which falls within the acceptable input voltage range. The Serial Clock Line (SCL) provides the timing reference for data transfer between the

microcontroller and the OLED display. The microcontroller generates this clock signal to synchronise communication. The Serial Data Line (SDA) is a bidirectional line that transfers address and data information between devices. The SCL and SDA pins are connected to the corresponding I2C pins on the MCU (PB8 and PB9 respectively).

3.8 SD Card

A generic SD card module is used as an adapter to connect the SD card to the MCU. The module has an operating voltage of 2.6 to 3.6V, supports a FAT file system, and has a quiescent current of 16 A (SD card is not inserted) [10]. The module uses an SPI interface (SPI3). Table 3 gives the SD card module's signal connections which enable communication with the MCU.

Module Signal	Connection	Description
3.3V	3.3V Power supply	Power input for module operation
Chip select (CS)	GPIO pin PD8	Activates communication with module
Master out slave in (MOSI)	GPIO pin PC12	Data transmission from MCU to module
Clock (CLK)	GPIO pin PB3	Synchronisation signal for data transfer
Master in slave out (MISO)	GPIO pin PC11	Data transmission from module to MCU
Ground (GND)	Ground (GND)	Common electrical reference point

Table 3: SD Card Module Signal Connections

3.9 RFID Reader

The RFID module used in this project is the RC522 RFID Development Kit, which uses the MFRC522 controller [11]. The RFID module uses an SPI interface (SPI1). Table 4 gives the RFID module's signal connections which enable communication with the MCU.

Module Signal	Connection	Description
Chip Select (CS) (labeled SDA)	GPIO pin PD9	Activate communication with module
Serial clock (SCK)	GPIO pin PA5	Synchronisation signal for data transfer
Master out slave in (MOSI)	GPIO pin PA7	Data transmission from MCU to module
Master in slave out (MISO)	GPIO pin PC11	Data transmission from module to MCU
Interrupt request (IRQ)	NA (unused)	Notify MCU of events without polling (unused)
Ground (GND)	Ground (GND)	Common electrical reference point
Reset (RST)	GPIO pin PC8	Initialise/restart module to known state
3.3V	3.3V Power supply	Power input for module operation

Table 4: RFID Module Signal Connections

3.9.1 Decoupling Capacitor

The RFID reader introduces electrical noise into the system due to its internal coil, which generates a radio frequency electromagnetic field to read RFID cards/tags. Powering this coil causes current changes that result in voltage fluctuations on the 3.3V power supply line. To mitigate this, a decoupling capacitor is connected between the module's power supply and ground, which helps in suppressing high-frequency noise to stabilise the voltage line.

4 Software Design and Implementation

This project makes use of STM32CubeIDE v1.16.1, with G0 Firmware package 1.6.2. GIT is used for source code version control. Code is written at the HAL level and the modularity of the software ensures that reworking of code is kept to a minimum. The system is designed to use a timer-based schedule inside the while loop with interrupts to make the software robust.

4.1 Software Block Diagram and Description of Interaction

Figure 9 shows the software block diagram for this project. Interfaces and modules are grouped by color. Text on the edges shows the data that flows between components.

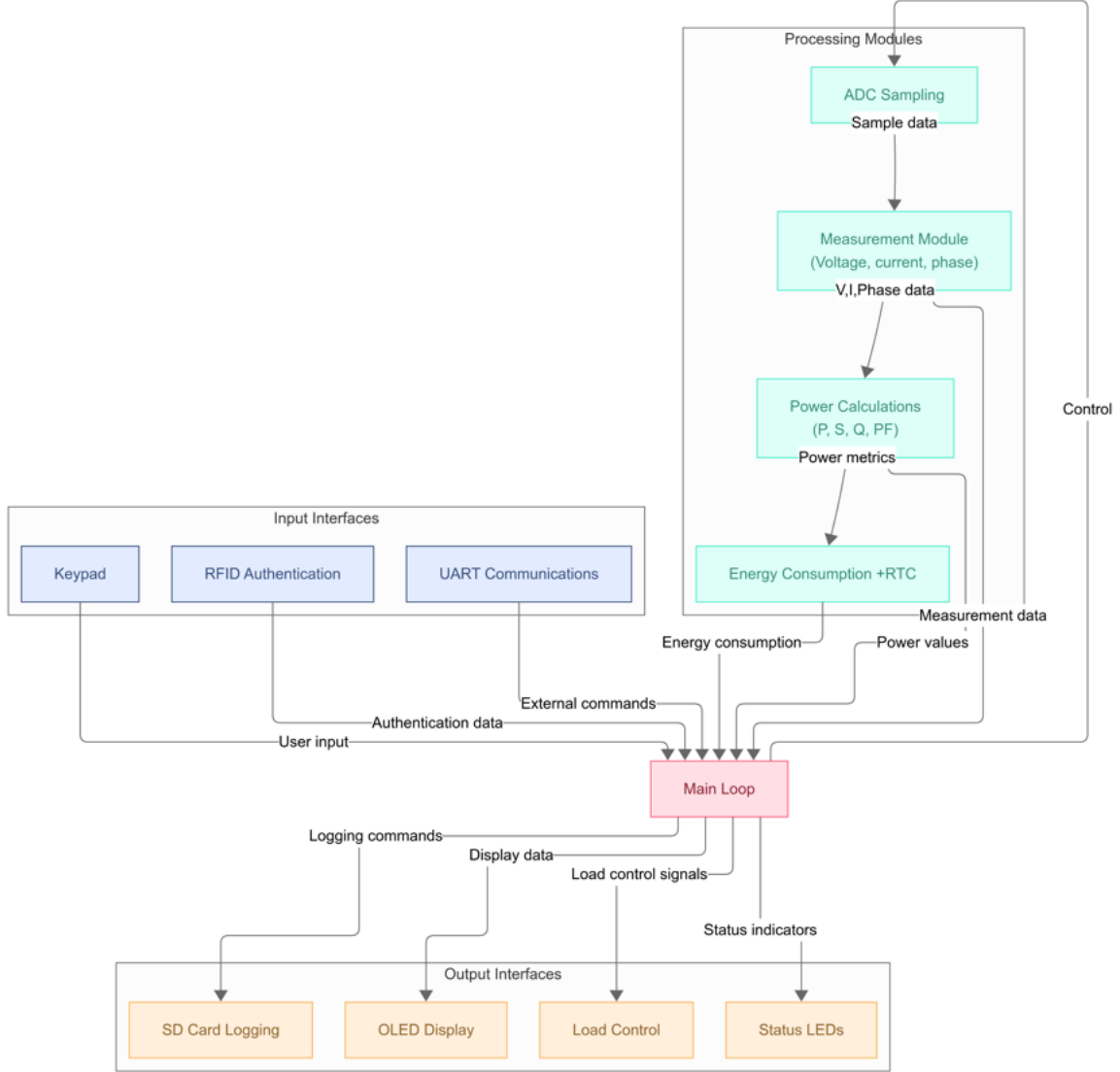


Figure 9: Software Block Diagram

4.2 ADC, Setup and Sampling Data Management

The MCU's internal ADC is used to measure the processed mains voltage, V_{mpp} , and processed mains current, I_{mpp} . Both signals are $50Hz$. The parameters to be measured are the amplitude of each signal as well as the phase difference between the signals.

The signal measurement subsystem relies on the coordinated operation of three MCU peripherals:

- Analog-to-Digital Converter (ADC1) which performs the actual signal sampling, converting analog voltage and current inputs into digital values.
- Timer2 (TIM2) which serves as the precise timing source, generating regular triggers at a fixed frequency of $5kHz$.
- Direct Memory Access (DMA) which automatically transfers conversion results from the ADC to memory without any CPU involvement.

Figure 10 shows how the ADC samples these signals.

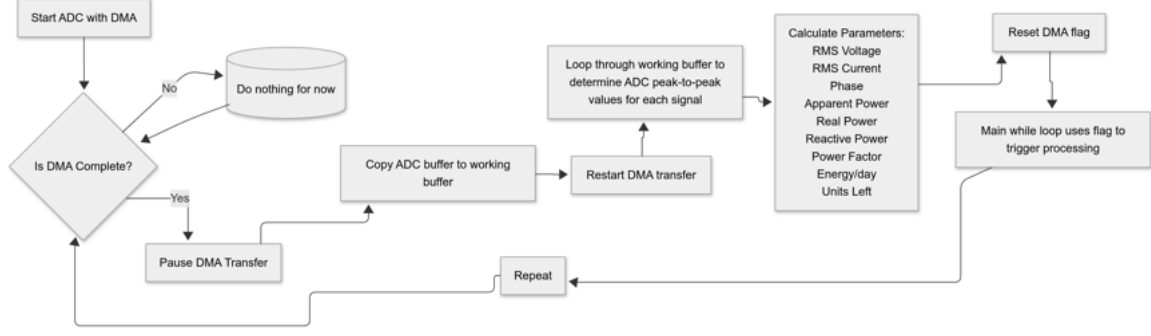


Figure 10: ADC Sampling Flow Chart

4.2.1 Setup

The following tables specify the initialisation/setup of components involved in the ADC operation. Only the (non-default) settings that are not listed in following sections appear in these tables. GPIO pin's PA0 and PA1 are enabled for ADC reading of the voltage and current, respectively. Table 5 lists the initialisation settings for the ADC, which are configured in the *.ioc* file.

Setting	Value	Description
Number of Conversions	2	Setup ADC to convert two channels (for two signals).
External Trigger Conversion Source	Timer 2 Trigger Out Event	Sets Timer 2's trigger output as the external trigger for ADC conversions.
External Trigger Conversion Edge	Trigger Detection on Rising Edge	Triggers conversions on the rising edge of the Timer 2 trigger signal.
Sampling Time Common 1/2	1.5 Cycles	Sets a short sampling time, suitable for low-impedance signals.

Table 5: ADC Initialisation Settings

Table 6 lists the initialisation settings for DMA, which are configured in the *.ioc* file.

Setting	Value	Description
DMA Request	ADC 1	Specifies source of DMA requests, triggering data transfers whenever new ADC conversion results are available.
Direction	Peripheral to Memory	This data flow direction allows automatic storage of conversion results.
Data Width	Half Word	DMA transfers 16-bit (half word) data elements, which maintains correct memory alignment.

Table 6: DMA Initialisation Settings

Timer 2 will generate update events at a frequency that is calculated using:

$$Frequency = \frac{SystemClock}{(Prescalar + 1) \times (CounterPeriod + 1)} \quad (8)$$

With the system clock set to $64MHz$, and substituting in the corresponding values:

$$Frequency = \frac{64MHz}{(12 + 1)(983 + 1)} = 5003Hz \approx 5kHz$$

4.2.2 Calibration

The signal measurement system requires calibration to ensure accurate voltage and current readings. The calibration process compensates for component tolerances, sensor characteristics, and ADC non-linearities. The signal measurement hardware incorporates a voltage divider and operational amplifier circuit before reaching the ADC. The system uses the following calibration constants:

- ADC Input Range: 0.3V to 3.0V (actual operating range).
- Operational Amplifier Gain: 1.33.
- Voltage Scale Factor (115): Converts sensor voltage to line voltage.
- Current Sensor Factor (7.5): Converts sensor output to current.

These constants were determined through a calibration procedure using a reference oscilloscope (and oscilloscope probe), performed across the operating range to ensure measurement accuracy within $\pm 5\%$ (Measurements section).

4.3 Analogue Measurements, Voltage, Current, and Phase

4.3.1 Measurement Process

The complete measurement process follows this sequence:

1. Timer2 runs continuously with a prescaler of 12 and counter period of 983. Based on the $64MHz$ system clock, this generates update events at approximately $5kHz$ (see Equation 8).
2. Each Timer2 update event triggers the ADC to begin a conversion sequence, ensuring precisely timed sampling intervals.
3. ADC1 is configured in scan mode to sample two channels sequentially, one for voltage and one for current. Each channel uses 12-bit resolution to provide adequate measurement precision.
4. When conversions complete, the ADC signals the DMA controller (with continuous requests enabled), which automatically transfers the results to a predefined memory buffer.
5. The DMA (channel 1) has priority 0 and is configured in circular mode, continuously filling a `uint32_t` buffer of size 400. Even indices (0, 2, 4...) store voltage samples while odd indices (1, 3, 5...) store current samples.
6. Because the sampling rate is approximately $5kHz$ and the signal frequency is $50Hz$, each cycle of the measured signal receives about 100 samples (50 voltage and 50 current samples).
7. The entire process operates autonomously once initialized, requiring no CPU intervention for data acquisition. This frees the processor to perform other tasks.

4.3.2 Processing Voltage and Current

The 12-bit ADC provides values between 0 – 4095, corresponding to an input range of 0.3 – 3.0V. The maximum peak-to-peak voltage/current is determined by finding the difference between maximum and minimum ADC values. A conversion formula is applied to transform raw ADC values to actual RMS values:

1. Voltage:

$$\begin{aligned} ADC_{pp} &= ADC_{max} - ADC_{min} \\ V_{pp} &= ADC_{pp} \times \frac{3.3}{4096} \text{ (converts ADC value to voltage)} \\ V_{RMS} &= \frac{V_{pp}}{1.33} \times 115 \text{ (scales measurement to actual mains voltage)} \end{aligned}$$

2. Current:

$$ADC_{pp} = ADC_{max} - ADC_{min}$$

$$I_{pp} = ADC_{pp} \times \frac{3.3}{4096} \text{ (converts ADC value to voltage)}$$

$$I_{RMS} = \frac{I_{pp}}{1.33} \times 7.5 \times 1000 \text{ (scales measurement to actual mains current in mA)}$$

4.3.3 Processing Phase

Phase is calculated using the zero-crossing method with interpolation. The maximum and minimum ADC values for voltage and current have already been determined, and can now be used to find the zero-crossing, which is effectively the midpoint, of each signal. Both voltage and current zero-crossings use Equation 9.

$$mid = min + \frac{max - min}{2} \quad (9)$$

The rising edge index at which each signal passes this point is then located, with interpolation. The unnormalised phase difference is found using Equation 10, where $sr = 5000Hz$ is the sampling rate. Finally, the unnormalised phase is normalised to between $-\pi/2$ and $\pi/2$.

$$\phi_{unnorm} = \frac{C_{cross} - V_{cross}}{sr} \times 50 \times 2 \times \pi \quad (10)$$

This entire phase calculation is shown by the flow chart in Figure 11.

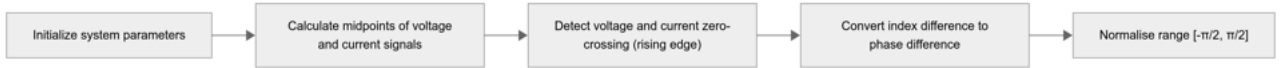


Figure 11: Phase Calculation Pseudo Code

4.4 Power Calculations, P, S, Q, and PF

Power calculations are made every time the ADC buffer fills up, i.e. every time the voltage, current, and phase measurements are updated. Thus, these parameters are updated at the end of the function that calculates phase. The reactive power is negative for a leading current and phase < 0 . The following formulas are used in power calculation (ϕ = phase):

- Apparent Power (VA): $S = V_{RMS} \times I_{RMS}$
- Real Power (W): $P = V_{RMS} \times I_{RMS} \times \cos(\phi)$
- Reactive Power (VAR): $Q = V_{RMS} \times I_{RMS} \times \sin(\phi)$
- Power Factor: $PF = \cos(\phi)$

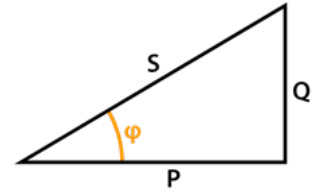


Figure 12: Power Triangle

4.5 UART Communications (Protocol, Data conversion)

The UART operates at a baud rate of 57600 bps, with 8 data bits, 1 parity bit, one stop bit, and even parity, using transistor-transistor-logic signal levels [5]. GPIO pins PA2 and PA3 are used at the UART transfer (Tx) and receive (Rx) pins, respectively. Upon power up, the board sends a student number (25964917) to the user which identifies the board. It then begins listening for messages. When a message is received, it is compared to the list of valid commands, and if the message matches a valid command, the board responds accordingly.

4.5.1 UART Protocol

UART is used to send and receive messages to and from the PC. This process is shown in Figure 13. The command descriptions, formats, and board responses are shown in Table 7 [5]. All of the PC commands starts with a '*' character and end with a '#' character followed by a new line ('\n') character. The values returned after receiving the Stat command are a list of parameters reporting the status of the system at that exact instance of time. Each of the lines returned are exactly 20 characters in length with the last character being a new line ('\n') character. The x's shown in Table 7 indicate the number format and required accuracy that is used.

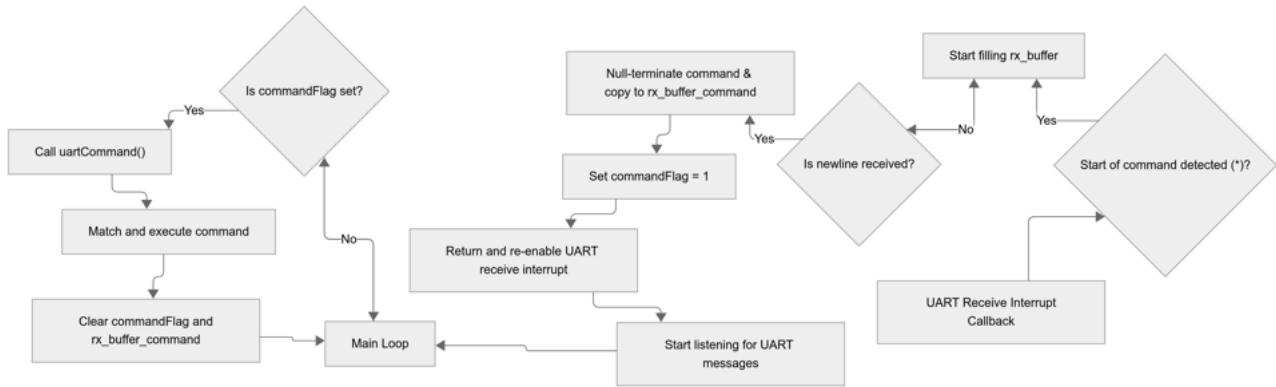


Figure 13: UART Tx and Rx Pseudo Code

Command description	Command received	Board response
Upon power up	None	*25964917#\n
Request SB status information	*Stat#\n	YYYY/MM/DD HH:MM:SS\n Voltage: xxx.xV\n Current: xxxxxmA\n Phase: x.xxxrad\n Apparent: xxxxxVA\n Real: xxxxxW\n Reactive: xxxxxVAR\n PowerFact: x.xxx\n Energy/d: xxxxxWh\n MaxPower: xxxxxW\n UnitsLeft: xxx.xxxkW\n
Toggle load switch ON/OFF	*Load#\n	No UART response
Toggle logging to SD Card ON/OFF	*Log#\n	No UART response
Dump SD card file contents over UART	*Dump#\n	Line-by-line information as stored in the .csv file
Clear file to erase SD card data	*CLF#\n	No UART response
Adds XXXXX units (Wh) to the system	*UnitsXXXXX#\n	No UART response

Table 7: UART TX Message fields - Log from board to PC

4.5.2 UART Timing

Upon system initialisation, 150 ms delay is introduced to allow the system to stabilise and then a student number is sent over UART2 using blocking transmission mode. The student number is

formatted as in Table 7. A function is then called to initialise UART2 to receive one byte using interrupt mode. It is called once, at startup, to begin listening for user commands. When a user command is received, it triggers an interrupt service routine. This interrupt service routine handles byte-by-byte reception over UART2. It accumulates bytes until a full command (terminated by "\n") is received. When a valid command is confirmed, the received command is copied into a second buffer to be handled appropriately. The UART2 receive buffer is reset, and once again, begins listening for commands. Recognised commands are listed in Table 7.

4.6 Keypad Driver Setup and Button Bounce Handling

The keypad software implements a low-level driver to read the key presses from the keypad. If the user presses a key, it closes a connection between a specific row and column, indicating which key was pressed. Each row is separately checked in time to enable the processor to read key presses. This is achieved by sequentially driving each row high (one at a time) and then "scanning" the columns to see if any of them are reading high (active high).

The process starts by setting all rows low, followed by a small delay to let the pins settle. Then each row (starting with row 1 on GPIO pin PA11, followed by row 2 on GPIO pin PB13, row 3 on GPIO pin PB10, and finally row 4 on GPIO pin PC0) is driven high one at a time while checking all three columns (column 1 on GPIO pin PC1, column 2 on GPIO pin PA12, and column 3 on GPIO pin PB2). If a column is detected as high while a specific row is driven high, the key at that row-column intersection is identified. The code only proceeds to check subsequent rows if no key press has been detected in the previous rows.

4.6.1 Button Bounce Handling

Button debouncing is necessary because mechanical buttons often produce multiple electrical signals when pressed due to mechanical bouncing of the contacts. Both the keypad buttons and the push button for emergency load control override are debounced in software. Both systems are configured as active-high and thus, use the same button de-bouncing logic, shown in Figure 14.

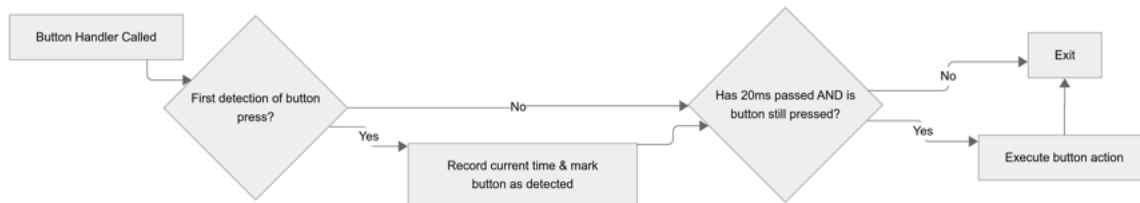


Figure 14: Software Button Debouncing Pseudo Code

4.7 OLED Low-Level Interface

The OLED software contains two levels of abstraction. The first and lower level is the driver, which is used to communicate with the OLED module at a low level and transfer commands and data to it via the I2C interface. The second and higher level is the application level, which is discussed in section 4.8. The OLED uses the SSD1306 graphic driver IC, an open-source driver [12] is used in this case, recommended by a Youtube tutorial [13].

4.8 FSM Control and Logic (Inputs: Keypad, UART. Outputs: LEDs, Load, OLED)

A Finite State Machine (FSM) is used to manage the information displayed on the OLED display. The system moves through the FSM based on user input received through the keypad. Figure 15 depicts the flow of the FSM according to user input, where a # cancels and goes back to the previous page. Table 18 [5] in the appendix gives a clearer breakdown of the OLED display menus.

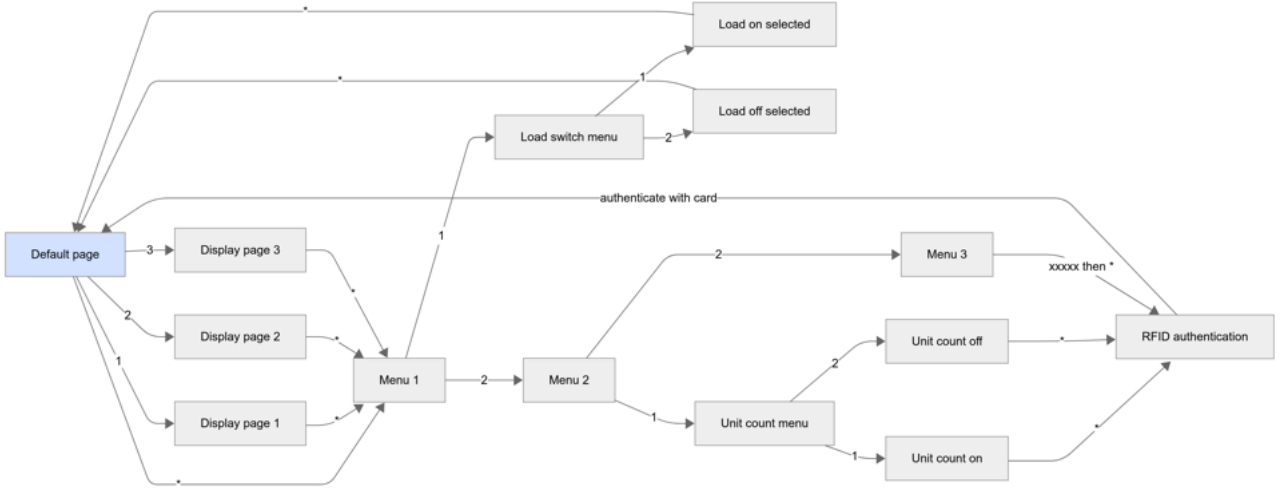


Figure 15: OLED Display FSM

Using UART, and the commands as mentioned in Section 4.5, a user is able to add units and manage the load switch. The output LED (set to output mode in the .ioc file) functionality, as discussed in Section 3.3, is realised according to the following criteria:

- LED D4: Toggles for each $1Wh$ decrease in energy units, checked with every power calculation.
- LED D3: Flashes every $500ms$ if units $< 10kWh$, monitored in the main while loop.
- LED D5: Flashes every $500ms$ if power usage $> 3kW$, monitored in the main while loop.

4.9 Energy Consumption Calculations + RTC

Energy consumption is calculated by subtracting the energy consumed over time (kWh) from the remaining energy units, using Equation 11. Eskom uses apparent power as effective power usage [14] apparent_power is multiplied by 0.04 (as this calculation is done every $40ms$) and divided by 3600000 (to convert seconds to hours, with a $40ms$ interval). The RTC ("HAL_GetTick()") is used for timing this $40ms$ interval. LED D4 is triggered to toggle, providing a visual indication of each $1Wh$ drop in energy, if units_left has decreased by at least $1Wh$.

$$\text{units_left} = \text{units_left} - \frac{\text{apparent_power} \times 0.04}{3600000} \quad (11)$$

4.10 SD Card Interface and Logging

4.10.1 Low-level interface

The MicroSD Card Module uses and SPI interface (SPI3), which is initialised exactly as done in this YouTube tutorial [15]. The module has a FAT file system requirement, thus, the middleware and software pack settings are configured to be FATFS. The YouTube tutorial makes use of a collection of imported, open-source input/output FATFS programs, from a GitHub repository [16], and these same programs are used in this project's software.

4.10.2 Logging

The SD card module is used to log electrical consumption data for the developed device. When the board loses power, the system is able to maintain critical information to ensure continued and uninterrupted operation. Each line of data logged to the SD-card is a comma-separated-values (.csv) file with 1 sec intervals between line entries. Table 8 shows the data fields that are logged, as well as the format that they are logged in [5].

Date and time	Voltage (V)	Current (mA)	Phase (rad)	Units left (kWh)
YYYY/mm/dd HH:mm:ss	xxx.x	xxxxx	x.xxx	xxx.xxx

Table 8: SD Card Log Format

Figure 16 shows software flow chart for the SD card logging function, which is called from the main while loop once per second to log a single line of data to the .csv file called "text.csv".

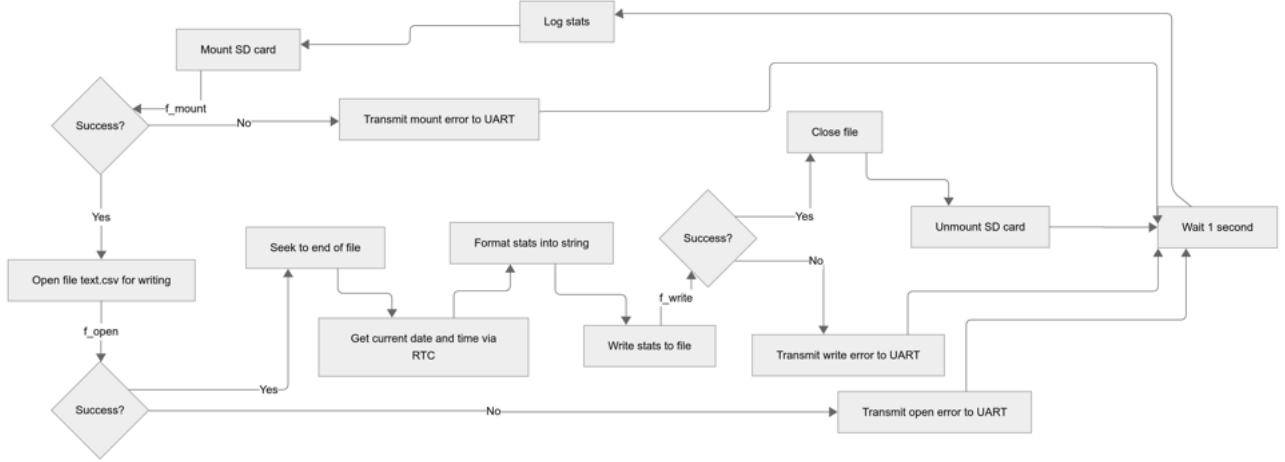


Figure 16: SD Card Logging Pseudo Code

4.11 RFID Interface and Authentication

The RFID Module uses an SPI interface (SPI1), which is initialised exactly (besides the different pin configurations, as discussed Section 3.9) as done in this YouTube tutorial [17]. The YouTube tutorial makes use of the authors own open-source library, which can be downloaded from a GitHub repository [18], and this same library is used in this project's software.

The RFID Module is initialised on power up. A function, `authorise()`, is called from the corresponding FSM states, when RFID authorisation is required. The states that require authorisation are given in Table 18 in the appendix. Figure 17 shows how RFID authorisation is achieved.

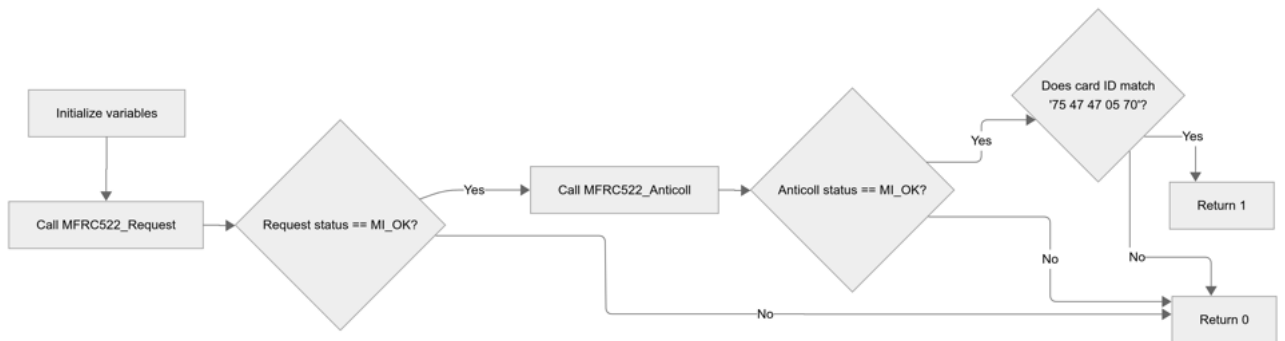


Figure 17: RFID Authorisation Pseudo Code

5 Measurements and Results

The modular design allows for independent testing of each subsystem.

5.1 Power supply

The 5V and 3.3V output rails were measured with various input voltages, using an oscilloscope probe.

5.1.1 5V Regulator

The measurement setup is shown in Figure 18. The results are summarised in Table 9, with the output voltages (V) given in columns 2 and 3. The 5V regulator achieves $\pm 5\%$ tolerance in all cases, which meets the design specification given [5]. The 5V regulator is guaranteed to work for an input of 9V - 12V, and the input should not exceed 13V, as the regulator was not tested above this input voltage and the behavior thereof is unknown (although as per [1], the maximum input voltage is 35V).

Input (V)	Load off	Load on
4.03	1.30	NA
5.05	2.87	NA
6.01	3.68	NA
7.06	4.8	4.6
8.04	5.03	5.03
9.02	5.03	5.03
10.01	5.03	5.03
11.03	5.03	5.03
12.01	5.03	5.03
13.03	5.02	5.02

Table 9: 5V Regulator Outputs

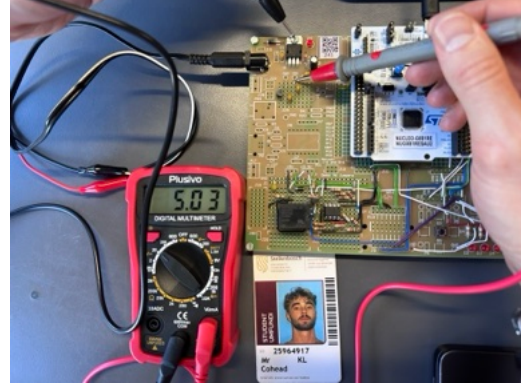


Figure 18: 5V Regulator Measurement Setup

5.1.2 3.3V Regulator

The measurement setup is shown in Figure 19. The results are summarised in Table 10, with the output voltages (V) given in columns 2, 3 and 4. The 3.3V regulator achieves $\pm 5\%$ tolerance in all cases, which meets the design specification given [5]. The 3.3V regulator is guaranteed to work for an input of 4.6V - 7V, and the input should not exceed 7V, as the regulator was not tested above this input voltage and the behavior thereof is unknown (although as per [2], the maximum input voltage is 30V). Further, the input of the 3.3V regulator is the output of the 5V regulator, which, in this project, has been proven to produce a stable 5V output, assuming it receives a valid input.

Input (V)	No load	150 Ω	22 Ω
2.03	2.01	1.71	1.13
3.02	2.98	2.71	2.03
4.02	3.30	3.30	3.05
4.62	3.30	3.30	3.30
5.03	3.30	3.30	3.30
6.02	3.30	3.30	3.30
7.02	3.30	3.30	3.30

Table 10: 3.3V Regulator Outputs



Figure 19: 3.3V Regulator Measurement Setup

5.2 Voltage and Current Circuits

5.2.1 Circuit Measurement Setup

Equipment	Connection
Signal generator	Crocodile clips between ground and J13-7,8
Oscilloscope	Oscilloscope probe 1 and 2 (both calibrated)
Oscilloscope probe 1	Between ground and PA0
Oscilloscope probe 2	Between ground and J13-7,8

Table 11: Measurement Test Equipment

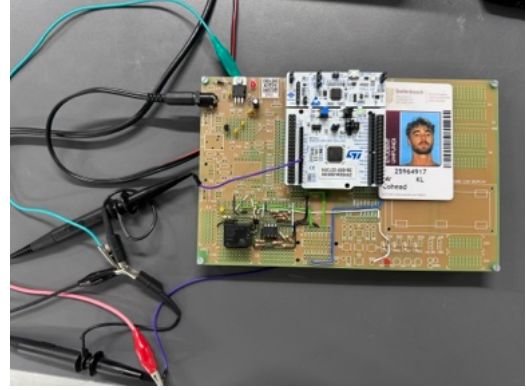


Figure 20: Circuit Measurement Setup

5.2.2 Circuit Measurement Test Results

Measured Input	Measured Output		Gain	
	V_{out}	I_{out}	V Gain	I Gain
$0.5V_{pp}$	$0.66V_{pp}$	$0.65V_{pp}$	1.32	1.30
$1V_{pp}$	$1.34V_{pp}$	$1.32V_{pp}$	1.34	1.32
$1.5V_{pp}$	$1.99V_{pp}$	$1.97V_{pp}$	1.33	1.33
$2V_{pp}$	$2.73V_{pp}$	$2.74V_{pp}$	1.37	1.37
$2.3V_{pp}$	$3.02V_{pp}$	$3.08V_{pp}$	1.31	1.34
Average Gain			1.334	1.332

Table 12: Voltage and Current Circuit Outputs and Gains

Live expressions are used in STM debug mode to display the values sampled by the ADC. Table 13 shows the results for the ADC voltage sampling accuracy, the ADC accuracy is the same for both current and voltage because the same channel is used for both.

ADC Input	Sampled ADC		Accuracy
	Values	Voltages	
$0.66V_{pp}$	1702 – 2527	$0.6646V_{pp}$	98.93%
$1.34V_{pp}$	1302 – 2956	$1.333V_{pp}$	99.50%
$1.99V_{pp}$	914 – 3369	$1.978V_{pp}$	99.39%
$2.73V_{pp}$	385 – 3719	$2.686V_{pp}$	98.36%
$3.02V_{pp}$	211 – 3959	$3.0196V_{pp}$	99.99%

Table 13: ADC Sampling Accuracy

5.2.3 Circuit Measurement Conclusions

The software uses a gain of 1.33 (as seen in step 3(a) of ADC Calibration) because the average measured gain is found to be 1.334 (voltage circuit) and 1.332 (current circuit), which are slightly different to the theoretically calculated gain of 1.35. This difference is likely due to the resistors used in the op-amp circuit, which have a tolerance of $\pm 5\%$. The voltage and current circuits are correctly implemented, with the system achieving well within $\pm 5\%$ accuracy for all tests. The oscilloscope (and probe) and ADC read slightly different values, which is to be expected. This is because the ADC is not as accurate as the oscilloscope (and probe).

5.3 Load Circuit

Figure 22 shows LED D2 illuminated, which indicates that a load switch command has been sent, the yellow LED illuminated, beside the relay, which indicates that the relay has switched. An N-channel MOSFET is used to control the relay. As seen in Figure 22, the gate-to-source voltage, V_{GS} , is driven to 3.22V when the relay is activated, and 0V when the relay is off, as seen in Figure 21. Since 3.22V exceeds the threshold voltage for this MOSFET (3V [9]), this is sufficient to fully turn the MOSFET on.



Figure 21: Load Off



Figure 22: Load On

5.4 Keypad and Button

To evaluate the behavior of the emergency load override button, the signal from the button was observed using an oscilloscope and oscilloscope probe. The output was triggered on a rising edge. Figure 23 shows the voltage signal which results from a button press. The signal exhibits a noticeable contact bounce, as expected. This bounce appears as multiple rapid voltage fluctuations within a short period ($\approx 5\text{ms}$). This bounce duration validates the need for software debouncing. In this project, the MCU waits 20ms before registering a valid press/release, after a trigger event.



Figure 23: Button Bounce



Figure 24: Keypad Bounce

To evaluate the behavior of the keypad, the signal from a key was observed using an oscilloscope and oscilloscope probe, with the output once again triggered on a rising edge. Figure 24 shows the voltage signal which results from a key press. A clear contact bounce can be observed. This bounce appears as multiple rapid voltage fluctuations within a short period ($\approx 15\text{ms}$). This bounce duration validates the need for software debouncing. Similarly to the emergency override button, the MCU waits 20ms before registering a valid press/release, after a trigger event on the keypad.

5.5 UART Communications

Upon power-up, a student number is sent via UART using the prescribed message format. This must be done no sooner than $100ms$ after power up and no later than $500ms$ after power-up. Figure 25 shows the oscilloscope display of two channel readings from the MCU. Channel 2 is the power supply and channel 1 is the transmission line.

When signal 2 goes high, at time $t_0 = 0ms$, a spike in signal 1 can be seen, followed by some power-on transient noise (internal oscillator/clock starting up, voltage regulator stabilisation, decoupling capacitors charging, etc...). The power-on transient noise lasts approximately $75ms$. At time $t_1 \approx 75ms$, signal 1 stabilises. At time $t_2 \approx 225ms$, signal 1 spikes low, which is the student number being sent via UART.

It can therefore be concluded that the student number is indeed sent, via UART, no sooner than $100ms$ after power up and no later than $500ms$ after power-up.



Figure 25: Power-up Student Number Transmission

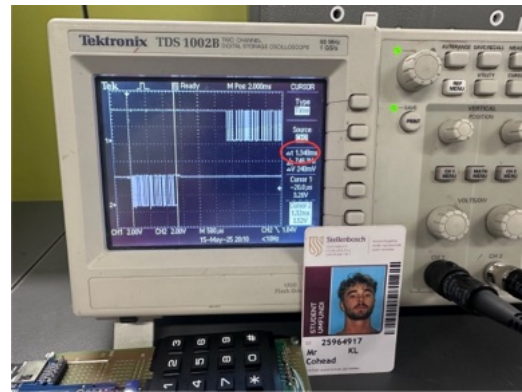


Figure 26: TX and RX Signals

Figure 26 shows the transfer (TX) and receive (RX) signals when using UART. These lines were tested with a `*Stat#` command. Channel 2 shows the RX line receiving the command and channel 1 shows the TX line transferring the statistics back via UART. The `*Stat#` command is 77 bits (with the new line character), and with a baud rate of 57600 bits/s, using Equation 12, the command should take $\approx 1.34ms$ which matches what can be seen in this figure (time for receiving of command circled in red), indicating that UART timing is accurate and functions as expected.

$$transmission\ time = bits \times \frac{1}{baud\ rate} \quad (12)$$

5.6 V, I, Phase, P, S, Q (incl. Calibration)

Live expressions are used in STM debug mode to display the sampled RMS voltage and current, phase, real power, apparent power, and reactive power.

5.6.1 Calibration

The signal generator is not a perfect piece of equipment, so the theoretical outputs (on signal generator screen) may not be the actual output of the signal generator. Thus, the inputs (signal generator outputs) for the measurements that follow in this section are measured and calibrated to ensure maximum accuracy. This is done by using an oscilloscope (and oscilloscope probe) to measure the signal before using it as an input for the system, and making the appropriate adjustments.

5.6.2 Voltage and Current RMS Measurements

The expected RMS values for voltage and current are compared against the actual sampled RMS values, with different inputs, to calculate the measurement functionality accuracy. The results are summarised in Table 14.

Input	Expected RMS		Sampled RMS		RMS Accuracy	
	V (V)	I (mA)	Voltage (V)	Current (mA)	Voltage	Current
$0.5V_{pp}$	57.5	3750	57.47	3876	99.93%	96.74%
$1V_{pp}$	115	7500	115.26	7592	99.77%	98.79%
$1.5V_{pp}$	172.5	11250	171.03	11276	99.15%	99.77%
$2V_{pp}$	230	15000	232.25	15201	98.68%	99.32%
$2.3V_{pp}$	264.5	17250	61.09	17262	98.71%	99.93%

Table 14: RMS Voltage and Current Accuracy

The expected RMS values and calculated RMS values differ slightly, which is a result of varying ADC readings, although $\pm 5\%$ accuracy is achieved for all tested input values.

5.6.3 Phase Measurements

Phase measurement accuracy was tested by varying the current phase. The maximum phase difference is defined as $\pm \pi/2$, thus, the maximum allowable difference between actual and measured phase is $\pm \pi/40 = \pm 4.5^\circ$. Table 15 summarises the results and the average accuracy meets the $\pm 5\%$ threshold.

Actual Phase (rad)	Measured Phase (rad)	Difference (abs)
-1.5	-1.471	0.029
-1	-0.983	0.017
-0.5	-0.481	0.019
0	0.010	0.010
0.5	0.484	0.016
1	0.994	0.006
1.5	1.493	0.007
Average Accuracy		96.93%

Table 15: Phase Measurement Accuracy

5.6.4 Power Measurements

The real, apparent, and reactive power are calculated based on the measured RMS voltages (V), RMS currents (mA), and phases (rad). Thus, using a sampled RMS measurements from Table 15 and the most inaccurate measured phase from Table 14, Table 16 summarises the power measurement accuracy. The power measurements meet the accuracy threshold in [5].

	Voltage	Current	Phase	Real (W)	Apparent (VA)	Reactive (VAR)
Expected	172.5	11250	-0.5	1703.06	1940.623	-930.39
Sampled	171.03	11276	-0.481	1709.71	1928.53	-892.28
Accuracy				99.96%	99.37%	95.90%

Table 16: Power Measurement Accuracy

5.7 OLED (Functional)

The OLED display functions as expected, as seen in Figure 27 and Figure 28. 2 different menu screens are shown to confirm that the OLED display updates and formats correctly, depending on the menu state.



Figure 27: OLED Display Functionality 1



Figure 28: OLED Display Functionality 2

5.8 SD Card (Functional)

The SD card functionality was tested by logging to the SD card for 5 seconds, dumping the SD card contents via UART, toggling power off and back on, dumping the SD card contents via UART again, verifying, via a stat command, that the units left parameter was initialised to what it was before power off, clearing the contents of the SD card, and dumping the SD card contents via UART one last time to confirm that the SD card was in fact cleared. Figure 29 shows the procedure.

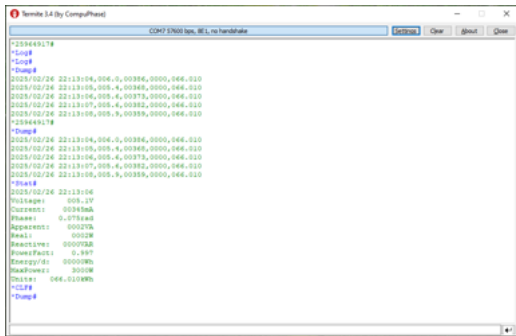


Figure 29: SD Card Functionality with UART

The student number represents the board powering on, units were then added via the keypad to change the default value. Statistics were logged for 5 seconds, the SD card contents were then dumped. Power was toggled off and on, indicated by the second student number. The second dump shows that the SD card retained its contents after losing power. The stat command shows that the units left parameter was initialised to the correct value after losing power (the most recent value on SD card). The SD card contents were then cleared, and the final dump command shows no response because the SD card contents were indeed deleted.

5.9 RFID (Functional)

The RFID functionality was tested using STM debug mode with live expressions. Figure 30 is a screenshot of variable "rfidNum", which stores the card ID, when the RFID module is polling, but no card is present. The variable is holding a random sequence. Figure 31 is a screenshot of variable "rfidNum", when the RFID module has scanned a card (117 71 71 5 112). "rfidNum" now stores the card ID which can be processed in software to confirm that it matches the authentication key.

Name	Type	Value
rfidStatus	uint8_t	2 '\002'
rfidStr	uint8_t [16]	0x20023fcc
rfidNum	uint8_t [5]	0x20023fc4
rfidNum[0]	uint8_t	241 'A'
rfidNum[1]	uint8_t	118 'V'
rfidNum[2]	uint8_t	0 '\0'
rfidNum[3]	uint8_t	8 '\b'
rfidNum[4]	uint8_t	255 'y'

Figure 30: Scanning Without Card

Name	Type	Value
rfidStatus	uint8_t	0 '\0'
rfidStr	uint8_t [16]	0x20023fcc
rfidNum	uint8_t [5]	0x20023fc4
rfidNum[0]	uint8_t	117 'u'
rfidNum[1]	uint8_t	71 'G'
rfidNum[2]	uint8_t	71 'G'
rfidNum[3]	uint8_t	5 '\005'
rfidNum[4]	uint8_t	112 'p'

Figure 31: Scanned Card

The RFID functions as expected, although the RFID antenna causes noise on the 3.3V supply rail, and disturbs the measurement system. In order to lessen the impact that the antenna has on the

system stability, it is only switched on when authentication is requested, and is switched back off as soon as authentication has completed, or has timed out.

5.10 Menu Functionality

Menu functionality was tested by confirming that both the correct menu path options are available and that the correct menu content is displayed. A full traversal of the FSM was conducted, moving through all content pages, splitting at nodes where options are available. Figure 32 shows this FSM traversal, and confirms the correct implementation thereof.

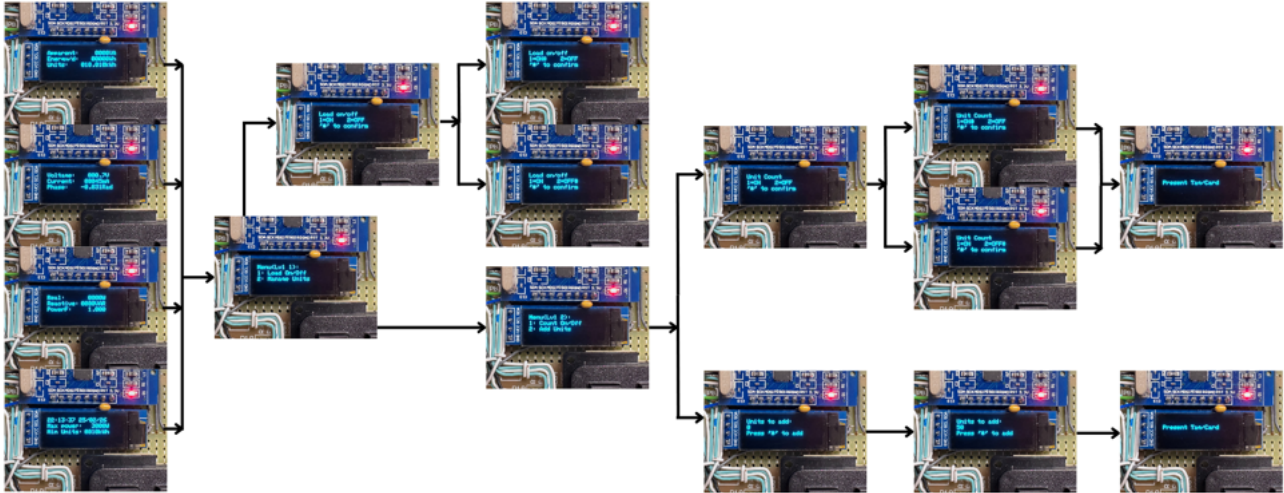


Figure 32: Full FSM Traversal

5.11 Full System (reliable, consistent, stress testing)

5.11.1 Circuit Clipping

The reliability of the voltage and current circuits is adequate. As seen in Figure 33, the voltage circuit exhibits very slight clipping with an input of $2.2V_{pp}$ (will not be exceeded). In Figure 34, the circuit clips at $2.3V_{pp}$. The current circuit produced identical results, as they are the same circuit, and respective conditioning is performed in software, thus, the current clipping test is excluded for brevity.

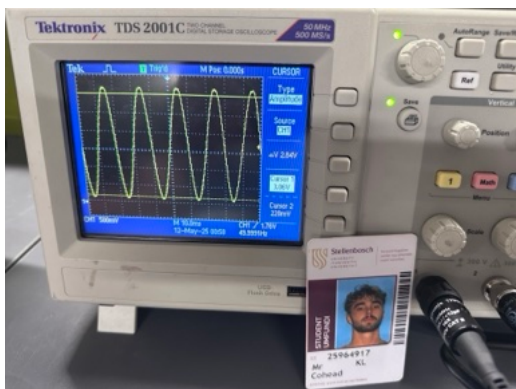


Figure 33: Voltage Circuit Slight Clipping



Figure 34: Voltage Circuit Clipping



Figure 38: LED D4 4.5s Toggling



Figure 39: LED D5 500ms Toggling

5.11.5 Circuit Performance

The voltage and current circuit inputs and outputs were measured using an oscilloscope (and oscilloscope probe) to confirm that the correct scaling and offset was being applied. Figure 40 shows the results for the voltage circuit (current excluded for brevity, as it looks identical). An input of $2V_{pp}$ produces an output of $\approx 2.7V_{pp}$, offset by $\approx 0.3V$, which confirms correct op-amp circuit performance.

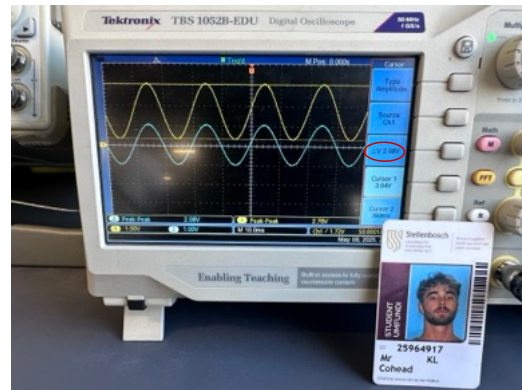


Figure 40: Voltage Circuit Input (blue) and Output (yellow)

6 Conclusions

The conclusions are drawn from demonstration feedback and self-implemented testing.

6.1 Non-compliance

This project met almost all of the demonstration requirements. Firstly, a slight UART formatting issue, which has since been fixed, and was due to copying and pasting the formatted stat response from a pdf document, caused the testing station to crash before testing could begin in the third demonstration. Secondly, in demonstration four, the logging functionality did not work exactly as expected. The logs showed an incorrect number of lines when dumped to the PC. This is a result of a timing issue, as logs were not made at exact 1s intervals, and suggests that there may be some blocking code in the software.

6.2 Possible Improvements

The improvements that could be made to this project include a revision on the software implementation, with a focus on removing all instances of blocking code, in order to improve the timing. Although fairly well implemented, the functions and variables in the software are not optimally organised, and this decreases the software readability. This issue does not directly influence performance or functionality, but improving on this downfall may very well help sort out the timing issue. The hardware is implemented neatly, and it is easy to where components are physically connected, so no improvements need to be made in this sense. Although, the RFID reader could be tested in different positions, and moved to one where it influences the measurement subsystem least.

7 Appendix

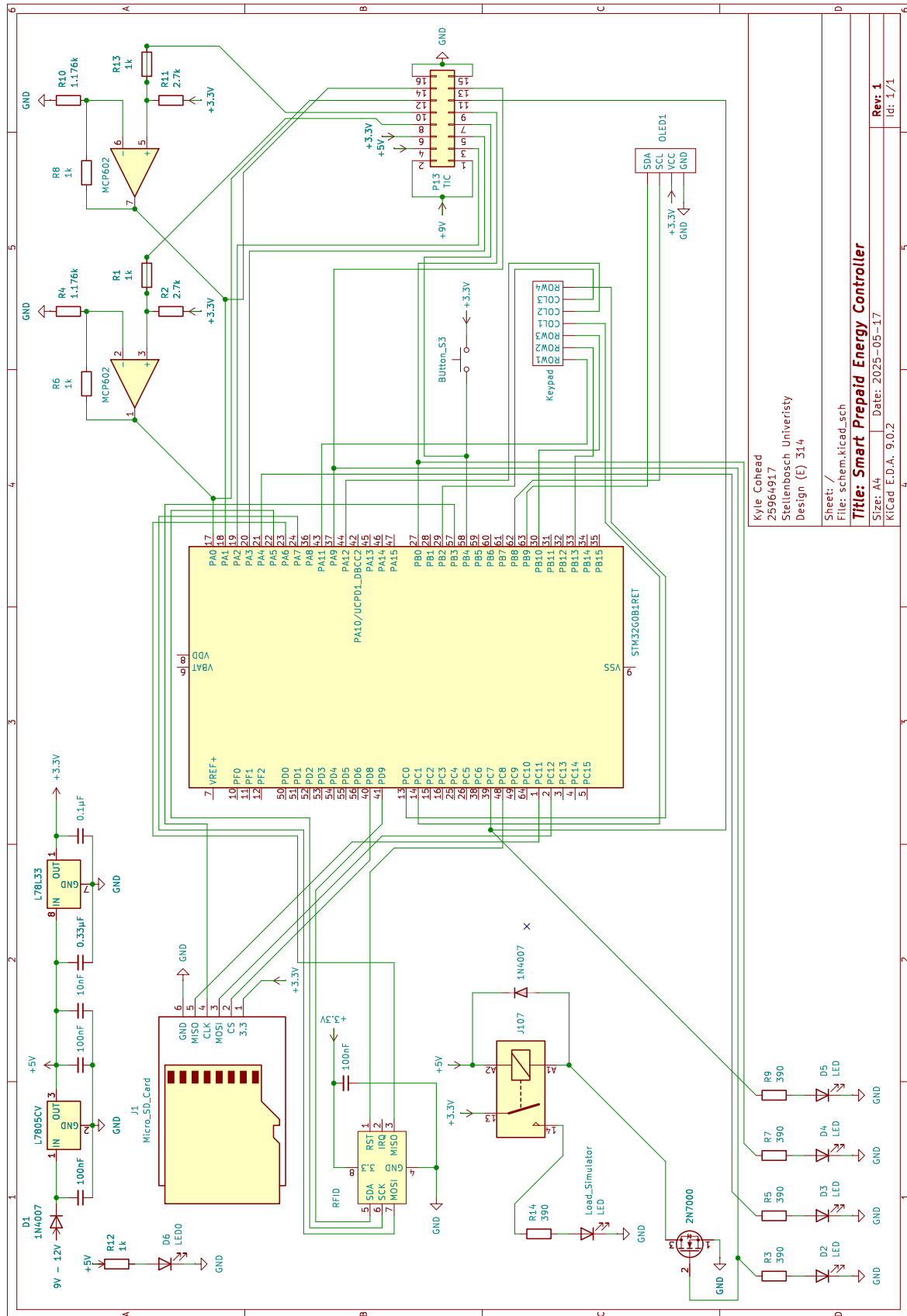


Figure 41: Project Schematic

STM32 Pin	Configuration
PA0	ADC1_IN0
PA1	ADC1_IN1
PA2	USART2_TX
PA3	USART2_RX
PA4	GPIO_Output
PA5	SPI1_SCK
PA6	SPI1_MISO
PA7	SPI1_MOSI
PA9	GPIO_Output
PA11	GPIO_Output
PA12	GPIO_EXTI2
PA13	SYS_SWDIO
PA14	SYS_SWCLK
PB0	GPIO_Output
PB2	GPIO_EXTI2
PB3	SPI3_SCK
PB4	GPIO_EXTI4
PB8	I2C1_SCL
PB9	I2C1_SDA
PB10	GPIO_Output
PB13	GPIO_Output
PC0	GPIO_Output
PC1	GPIO_EXTI1
PC7	GPIO_Output
PC8	GPIO_Output
PC11	SPI3_MISO
PC12	SPI3_MOSI
PC13	GPIO_EXTI13
PC14	RCC_OSC32_IN
PC15	RCC_OSC32_OUT
PD8	GPIO_Output
PD9	GPIO_Output

Table 17: STM32 Pins Used and thier Configuration

Keypad Press	Action	Display OLED
0	default page	Apparent: xxxxVA Energy/d: xxxxxWh Units: xxx.xxxkWh
1	display page 1	Voltage: xxx.xV Current: xxxxxmA Phase: x.xxxRad
2	display page 2	Real: xxxkW Reactive: xxxkVAR PowerFact: x.xxx
3	display page 3	HH:MM:SS YY/MM/DD Max power: xxxkW Min Units: xxxkWh
*	enter menu 1	Menu(Lv1 1): 1: Load On/Off 2: Manage Units
*	confirm setting	<i>If pressed setting is confirmed. Screen returns to Default page.</i>
#	cancel setting	<i>If pressed exits to the previous page.</i>
* > 1	Load switch menu	Load on/off 1=ON 2=OFF '*' to confirm
* > 1 > 1	Load on selected	Load on/off 1=ON# 2=OFF '*' to confirm
* > 1 > 2	Load off selected	Load on/off 1=ON 2=OFF# '*' to confirm
* > 2	menu 2	Menu(Lv1 2): 1: Count On/Off 2: Add Units
* > 2 > 1	Unit count menu	Unit Count on/off 1=ON 2=OFF '*' to confirm
* > 2 > 1 > 1	Count on selected	Unit Count on/off 1=ON# 2=OFF '*' to confirm
* > 2 > 1 > 2	Count off selected	Unit Count on/off 1=ON 2=OFF# '*' to confirm
* > 2 > 1 > x > *	RFID Authentication	Present Tag/Card
* > 2 > 2 > xxxxx	menu 3	Units to add: 0 (Note: The screen is awaiting units input xxx) Press '*' to add
* > 2 > 2 > xxx > *	RFID Authentication	Present Tag/Card

Table 18: OLED Display Screens for Various Input Combinations from Keypad.

References

- [1] *L7800 series positive voltage regulators*, STMicroelectronics, August 2006, accessed: 20 February 2025. [Online]. Available: [\url{chrome-extension://efaidnbmnnnibpcajpcgclefindmkaj/https://stemlearn.sun.ac.za/pluginfile.php/17755/mod_resource/content/2/STMicroelectronics-L7805CV-datasheet.pdf}](https://stemlearn.sun.ac.za/pluginfile.php/17755/mod_resource/content/2/STMicroelectronics-L7805CV-datasheet.pdf)
- [2] *L78LxxAB L78LxxAC - L78LxxC positive voltage regulators*, STMicroelectronics, April 2009, accessed: 20 February 2025. [Online]. Available: [\url{chrome-extension://efaidnbmnnnibpcajpcgclefindmkaj/https://stemlearn.sun.ac.za/pluginfile.php/163910/mod_resource/content/0/Regulator_L78l33.pdf}](https://stemlearn.sun.ac.za/pluginfile.php/163910/mod_resource/content/0/Regulator_L78l33.pdf)
- [3] *LED 3mm through hole RED*, Brightek (Eorope) Limited, accessed: 22 February 2025. [Online]. Available: [\url{chrome-extension://efaidnbmnnnibpcajpcgclefindmkaj/https://stemlearn.sun.ac.za/pluginfile.php/163912/mod_resource/content/0/LED_3mm_through_hole_RED.pdf}](https://stemlearn.sun.ac.za/pluginfile.php/163912/mod_resource/content/0/LED_3mm_through_hole_RED.pdf)
- [4] *STM32G0B1xB/xC/xE Data Sheet*, STMicroelectronics, June 2024, accessed: 20 February 2025. [Online]. Available: [\url{chrome-extension://efaidnbmnnnibpcajpcgclefindmkaj/https://stemlearn.sun.ac.za/pluginfile.php/133023/mod_resource/content/0/stm32g0b1re.pdf}](https://stemlearn.sun.ac.za/pluginfile.php/133023/mod_resource/content/0/stm32g0b1re.pdf)
- [5] A. Barnard and A. du Plessis, “Design (E) 314: Project Definition,” Course material for Design (E) 314, 2025.
- [6] *2.7V to 6.0V Single Supply CMOS Op Amps*, Microchip Technology Inc, 2007, accessed: 10 March 2025. [Online]. Available: [\url{https://stemlearn.sun.ac.za/pluginfile.php/17765/mod_resource/content/1/MCP602.pdf}](https://stemlearn.sun.ac.za/pluginfile.php/17765/mod_resource/content/1/MCP602.pdf)
- [7] B. Carter, *Designing Gain and Offset in Thirty Seconds*, Texas Instruments, February 2002, accessed: 10 March 2025. [Online]. Available: [\url{https://stemlearn.sun.ac.za/pluginfile.php/17757/mod_resource/content/0/sloa097-Designing%20Gain%20and%20Offset%20in%20Thirty%20Seconds.pdf}](https://stemlearn.sun.ac.za/pluginfile.php/17757/mod_resource/content/0/sloa097-Designing%20Gain%20and%20Offset%20in%20Thirty%20Seconds.pdf)
- [8] *J107 Relay Datasheet*, CIT, January 2023, accessed: 11 March 2025. [Online]. Available: [\url{chrome-extension://efaidnbmnnnibpcajpcgclefindmkaj/https://stemlearn.sun.ac.za/pluginfile.php/17758/mod_resource/content/2/J107%20Relay%20Datasheet.pdf}](https://stemlearn.sun.ac.za/pluginfile.php/17758/mod_resource/content/2/J107%20Relay%20Datasheet.pdf)
- [9] *N-Channel Enhancement-Mode Vertical DMOS FET*, Microchip Technology Inc, 2001, accessed: 11 March 2025. [Online]. Available: [\url{chrome-extension://efaidnbmnnnibpcajpcgclefindmkaj/https://stemlearn.sun.ac.za/pluginfile.php/17759/mod_resource/content/2/2n7000-n-channel-MOSFET.pdf}](https://stemlearn.sun.ac.za/pluginfile.php/17759/mod_resource/content/2/2n7000-n-channel-MOSFET.pdf)
- [10] A. Barnard and A. du Plessis, “SD card reader module – Datasheet,” Course material for Design (E) 314, 2025, accessed: 8 May 2025. [Online]. Available: [\url{chrome-extension://efaidnbmnnnibpcajpcgclefindmkaj/https://stemlearn.sun.ac.za/pluginfile.php/17761/mod_resource/content/2/SD%20card%20reader%20Datasheet.pdf}](https://stemlearn.sun.ac.za/pluginfile.php/17761/mod_resource/content/2/SD%20card%20reader%20Datasheet.pdf)
- [11] *RFID RC522 Dev Kit Datasheet*, Handson Technology, accessed: 10 May 2025. [Online]. Available: [\url{chrome-extension://efaidnbmnnnibpcajpcgclefindmkaj/https://stemlearn.sun.ac.za/pluginfile.php/17756/mod_resource/content/1/RFID%20RC522%20Dev%20Kit%20Datasheet.pdf}](https://stemlearn.sun.ac.za/pluginfile.php/17756/mod_resource/content/1/RFID%20RC522%20Dev%20Kit%20Datasheet.pdf)
- [12] A. Fiskov, “stm32-ssd1306: STM32 library for working with OLEDs based on SSD1306, SH1106, SH1107 and SSD1309, supports I2C and SPI,” <https://github.com/afiskon/stm32-ssd1306>, 2023, accessed: 11 April 2025.
- [13] hacksontable, “STM32 OLED TUTORIAL,” YouTube, April 2022, accessed: 11 April 2025. [Online]. Available: <https://www.youtube.com/watch?v=videoID>

- [14] *A Guide to Understanding Your Industrial Electricity Bill*, National Cleaner Production Centre South Africa, accessed: 16 May 2025. [Online]. Available: <https://www.ncpc.co.za/resource-efficiency-and-cleaner-production-a-guide-to-understanding-your-industrial-electricity-bill.pdf>
- [15] PR TechTalk, “STM32 SD Card Interface via SPI and FatFS Tutorial (No RTOS),” YouTube, October 2024, accessed: 9 May 2025. [Online]. Available: <https://www.youtube.com/watch?v=PBIIm8BCnbyQ>
- [16] H. Pearce, TheFaraDos, S. Martel, and A. Wojasiński, “cubeide-sd-card: This project is designed as an example of a STM32CubeIDE-generated system with FatFs middleware controlling an SPI-connected MMC/SD memory card,” <https://github.com/kiwiwh/cubeide-sd-card>, 2023, accessed: 10 May 2025.
- [17] CircuitGator HQ, “How to use RFID RC522 with STM32,” YouTube, January 2025, accessed: 11 May 2025. [Online]. Available: <https://www.youtube.com/watch?v=Ay3ysGDrxbo>
- [18] C. HQ, “RFID-RC522-with-STM32,” <https://github.com/CircuitGatorHQ/RFID-RC522-with-STM32>, 2025, accessed: 11 May 2025.